

Язык программирования C++

Лекция 18: Приведение типов, RTTI, typeid, lvalue/rvalue, move, вывод типов, range-based for

Никита Лисица

2026

Напоминание: приведение типов в C++

- Указатели автоматически приводятся к `void*`, к константной версии того же указателя, и к указателю на родительский класс:

```
int i = 12;
void* p = &i; // OK
const int* q = &i; // OK

struct A {};
struct B : A {};

B b;
A* pa = &b; // OK
```

Напоминание: приведение типов в C++

- В других случаях будет ошибка компиляции:

```
void* p = ...;
int* q1 = p; // Ошибка компиляции
int* q2 = (int*)p; // ОК, явное приведение

const int i = 12;
int* pi1 = &i; // Ошибка компиляции
int* pi2 = (int*)&i; // ОК, явное приведение

A a;
B* pb1 = &a; // Ошибка компиляции
B* pb2 = (B*)&a; // ОК, явное приведение
```

Напоминание: приведение типов в C++

- Все числовые типы приводятся друг к другу:

```
int i = 12;  
float f = i;  
double d = f;  
char c = d;  
std::uint32_t u = f;
```

Напоминание: приведение типов в C++

- Все числовые типы приводятся друг к другу:

```
int i = 12;  
float f = i;  
double d = f;  
char c = d;  
std::uint32_t u = f;
```

- NB: часть таких приведений – *undefined behavior*

Напоминание: приведение типов в C++

- Для приведения своих типов (структур/классов) можно сделать неявный конструктор или оператор приведения:

```
class BigInteger {  
public:  
    BigInteger(int value); // Приведение int -> BigInteger  
    operator int() const; // Приведение BigInteger -> int  
};
```

Напоминание: приведение типов в C++

- Для приведения своих типов (структур/классов) можно сделать неявный конструктор или оператор приведения:

```
class BigInteger {  
public:  
    BigInteger(int value); // Приведение int -> BigInteger  
    operator int() const; // Приведение BigInteger -> int  
};
```

- **NB:** если для приведения типов $A \rightarrow B$ есть как оператор приведения $A::\text{operator } B()$, так и конструктор $B::B(A)$, будет ошибка компиляции

- Конструкция `(int*)p` или `(float)42` или `(int)myBigInt` называется *приведением типов в стиле C* (*C-style cast*)

Приведение типов в C++

- Конструкция `(int*)p` или `(float)42` или `(int)myBigInt` называется *приведением типов в стиле C* (*C-style cast*)
- В C++ принято её не использовать, так как она умеет слишком многое, и плохо выражает намерение программиста

Приведение типов в C++

- Конструкция `(int*)p` или `(float)42` или `(int)myBigInt` называется *приведением типов в стиле C* (*C-style cast*)
- В C++ принято её не использовать, так как она умеет слишком многое, и плохо выражает намерение программиста
- Вместо этого, есть 4 специализированных оператора приведения типов, которые выглядят как шаблонные функции:

Приведение типов в C++

- Конструкция `(int*)p` или `(float)42` или `(int)myBigInt` называется *приведением типов в стиле C* (*C-style cast*)
- В C++ принято её не использовать, так как она умеет слишком многое, и плохо выражает намерение программиста
- Вместо этого, есть 4 специализированных оператора приведения типов, которые выглядят как шаблонные функции:
 - `static_cast<T>(x)`

Приведение типов в C++

- Конструкция `(int*)p` или `(float)42` или `(int)myBigInt` называется *приведением типов в стиле C* (*C-style cast*)
- В C++ принято её не использовать, так как она умеет слишком многое, и плохо выражает намерение программиста
- Вместо этого, есть 4 специализированных оператора приведения типов, которые выглядят как шаблонные функции:
 - `static_cast<T>(x)`
 - `reinterpret_cast<T>(x)`

Приведение типов в C++

- Конструкция `(int*)p` или `(float)42` или `(int)myBigInt` называется *приведением типов в стиле C* (*C-style cast*)
- В C++ принято её не использовать, так как она умеет слишком многое, и плохо выражает намерение программиста
- Вместо этого, есть 4 специализированных оператора приведения типов, которые выглядят как шаблонные функции:
 - `static_cast<T>(x)`
 - `reinterpret_cast<T>(x)`
 - `const_cast<T>(x)`

Приведение типов в C++

- Конструкция `(int*)p` или `(float)42` или `(int)myBigInt` называется *приведением типов в стиле C* (*C-style cast*)
- В C++ принято её не использовать, так как она умеет слишком многое, и плохо выражает намерение программиста
- Вместо этого, есть 4 специализированных оператора приведения типов, которые выглядят как шаблонные функции:
 - `static_cast<T>(x)`
 - `reinterpret_cast<T>(x)`
 - `const_cast<T>(x)`
 - `dynamic_cast<T>(x)`

- Всё, что умеют неявные приведения типов

- Всё, что умеют неявные приведения типов
- Приведение указателя на базовый класс к указателю на наследник (*downcasting*)

- Всё, что умеют неявные приведения типов
- Приведение указателя на базовый класс к указателю на наследник (*downcasting*)
- Приведение указателя на **void** к другому типу указателя

- Всё, что умеют неявные приведения типов
- Приведение указателя на базовый класс к указателю на наследник (*downcasting*)
- Приведение указателя на **void** к другому типу указателя

```
int i = static_cast<int>(3.14f);
```

```
A* pa = new B;
```

```
B* pb = static_cast<B*>(pa);
```

```
void* pv = pa;
```

```
A* pa2 = static_cast<A*>(pv);
```

static_cast

- Всё, что умеют неявные приведения типов
- Приведение указателя на базовый класс к указателю на наследник (*downcasting*)
- Приведение указателя на **void** к другому типу указателя

```
int i = static_cast<int>(3.14f);

A* pa = new B;
B* pb = static_cast<B*>(pa);

void* pv = pa;
A* pa2 = static_cast<A*>(pv);
```

- Легче увидеть в коде, понятнее намерения программиста

- Приведение между указателями любых типов

- Приведение между указателями любых типов
- Приведение между ссылками любых типов

- Приведение между указателями любых типов
- Приведение между ссылками любых типов
- Приведение между указателями и числами

reinterpret_cast

- Приведение между указателями любых типов
- Приведение между ссылками любых типов
- Приведение между указателями и числами

```
int* p = new int[100];  
float* f = reinterpret_cast<float*>(p);  
  
int& ri = p[10];  
float& rf = reinterpret_cast<float&>(ri);  
  
std::uint64_t address = reinterpret_cast<std::uint64_t>(p);  
int* q = reinterpret_cast<int*>(address);
```

reinterpret_cast

- Приведение между указателями любых типов
- Приведение между ссылками любых типов
- Приведение между указателями и числами

```
int* p = new int[100];  
float* f = reinterpret_cast<float*>(p);  
  
int& ri = p[10];  
float& rf = reinterpret_cast<float&>(ri);  
  
std::uint64_t address = reinterpret_cast<std::uint64_t>(p);  
int* q = reinterpret_cast<int*>(address);
```

- Обычно нужен в низкоуровневом коде

reinterpret_cast

- Приведение между указателями любых типов
- Приведение между ссылками любых типов
- Приведение между указателями и числами

```
int* p = new int[100];  
float* f = reinterpret_cast<float*>(p);  
  
int& ri = p[10];  
float& rf = reinterpret_cast<float&>(ri);  
  
std::uint64_t address = reinterpret_cast<std::uint64_t>(p);  
int* q = reinterpret_cast<int*>(address);
```

- Обычно нужен в низкоуровневом коде
- Нужно использовать с осторожностью

- Позволяет убрать/добавить константность у указателя или ссылки

- Позволяет убрать/добавить константность у указателя или ссылки

```
const int x = 42;  
int* p = &x; // Ошибка компиляции  
int* q = const_cast<int*>(&x); // OK  
*q = 67; // Undefined behavior
```

- Позволяет убрать/добавить константность у указателя или ссылки

```
const int x = 42;  
int* p = &x; // Ошибка компиляции  
int* q = const_cast<int*>(&x); // OK  
*q = 67; // Undefined behavior
```

- Нужно использовать с крайней осторожностью

- Позволяет убрать/добавить константность у указателя или ссылки

```
const int x = 42;  
int* p = &x; // Ошибка компиляции  
int* q = const_cast<int*>(&x); // OK  
*q = 67; // Undefined behavior
```

- Нужно использовать с крайней осторожностью
- Почти никогда не нужен, исключение – реализация парных **const** и не-**const** методов у классов

- Часто, класс предоставляет две версии метода (константную и неконстантную), которые практически не отличаются реализацией

- Часто, класс предоставляет две версии метода (константную и неконстантную), которые практически не отличаются реализацией

```
template <typename Key, typename Value>
class map {
    // ...

    const Value& at(const Key& key) const {
        // Поиск в дереве
    }

    Value& at(const Key& key) {
        // Точно такой же поиск в дереве :)
    }
};
```

- Возможное решение – использовать `const_cast`

- Возможное решение – использовать `const_cast`

```
template <typename Key, typename Value>
class map {
    // ...

    const Value& at(const Key& key) const {
        // Поиск в дереве
    }

    Value& at(const Key& key) {
        // Здесь const_cast безопасен: мы знаем, что объект *this
        // на самом деле неконстантный
        return const_cast<Value&>(
            const_cast<const map&>(*this).at(key)
        );
    }
};
```

- Безопасное приведение указателя/ссылки на базовый класс к указателю на родительский класс

dynamic_cast

- Безопасное приведение указателя/ссылки на базовый класс к указателю на родительский класс
- Если приведение некорректно, возвращается нулевой указатель, либо (в случае приведения типов ссылок) бросается исключение `std::bad_cast`

dynamic_cast

- Безопасное приведение указателя/ссылки на базовый класс к указателю на родительский класс
- Если приведение некорректно, возвращается нулевой указатель, либо (в случае приведения типов ссылок) бросается исключение `std::bad_cast`

```
struct A {  
    virtual ~A() {}  
};  
  
struct B : A { ... };  
  
struct C : A { ... };  
  
A * pa = new B;  
  
dynamic_cast<B*>(pa); // вернёт реальный указатель  
dynamic_cast<C*>(pa); // вернёт нулевой указатель  
  
dynamic_cast<B&>(*pa); // вернёт реальную ссылку  
dynamic_cast<C&>(*pa); // бросит std::bad_cast
```

- Как работает `dynamic_cast`?

Run-Time Type Information (RTTI)

- Как работает `dynamic_cast`?
- Переходит по указателю на таблицу виртуальных функций, рядом с ней компилятор кладёт метаинформацию о типе

Run-Time Type Information (RTTI)

- Как работает `dynamic_cast`?
- Переходит по указателю на таблицу виртуальных функций, рядом с ней компилятор кладёт метаинформацию о типе
- $\implies$ `dynamic_cast` работает только для классов с виртуальными методами

Run-Time Type Information (RTTI)

- Как работает `dynamic_cast`?
- Переходит по указателю на таблицу виртуальных функций, рядом с ней компилятор кладёт метаинформацию о типе
- $\implies$ `dynamic_cast` работает только для классов с виртуальными методами
- Такая технология называется *Run-Time Type Information (RTTI)*

Run-Time Type Information (RTTI)

- Как работает `dynamic_cast`?
- Переходит по указателю на таблицу виртуальных функций, рядом с ней компилятор кладёт метаинформацию о типе
- $\implies$ `dynamic_cast` работает только для классов с виртуальными методами
- Такая технология называется *Run-Time Type Information (RTTI)*
- При программировании под микроконтроллеры или встраиваемые устройства её часто отключают (у GCC флаг `-fno-rtti`)

- В стандартной библиотеке есть специальный класс `std::type_info` в файле `<typeinfo>`

- В стандартной библиотеке есть специальный класс `std::type_info` в файле `<typeinfo>`
- Он позволяет узнать немного информации о типе объекта – вывести имя (*после name mangling'a!*) или посчитать хеш

- В стандартной библиотеке есть специальный класс `std::type_info` в файле `<typeinfo>`
- Он позволяет узнать немного информации о типе объекта – вывести имя (*после name mangling'a!*) или посчитать хеш
- Чтобы достать `std::type_info` по конкретному объекту или типу, есть оператор `typeid`

- В стандартной библиотеке есть специальный класс `std::type_info` в файле `<typeinfo>`
- Он позволяет узнать немного информации о типе объекта – вывести имя (*после name mangling'a!*) или посчитать хеш
- Чтобы достать `std::type_info` по конкретному объекту или типу, есть оператор `typeid`

```
#include <typeinfo>
#include <iostream>

int main() {
    int i = 10;
    std::cout << typeid(i).name() << std::endl;
    std::cout << typeid(std::ostream&).name() << std::endl;
}
```

- Если не выключен RTTI, и тип имеет виртуальные функции, то `typeid` будет использовать RTTI и вернёт настоящий тип объекта:

```
struct A {  
    virtual ~A() {}  
};  
  
struct B : A {};  
  
A * pa = new B;  
typeid(*pa); // вернёт typeid(B)
```

- `std::type_info` – неудобный (некопируемый) тип с реализацией, зависящей от компилятора

- `std::type_info` – неудобный (некопируемый) тип с реализацией, зависящей от компилятора
- Чтобы было удобнее хранить такие объекты в коллекциях, есть класс-обёртка `std::type_index`

- `std::type_info` – неудобный (некопируемый) тип с реализацией, зависящей от компилятора
- Чтобы было удобнее хранить такие объекты в коллекциях, есть класс-обёртка `std::type_index`
- Фактически, он просто хранит указатель на `std::type_info`

- `std::type_info` – неудобный (некопируемый) тип с реализацией, зависящей от компилятора
- Чтобы было удобнее хранить такие объекты в коллекциях, есть класс-обёртка `std::type_index`
- Фактически, он просто хранит указатель на `std::type_info`

```
std::set<std::type_index> set_of_types;  
set_of_types.insert(typeid(int));  
set_of_types.insert(typeid(void*));  
set_of_types.insert(typeid(std::string));
```

- Если хотите получить нормальное имя типа (до name mangling'a, т.н. *demangled name*), в библиотеке Boost есть функция, которая это делает:

Name mangling

- Если хотите получить нормальное имя типа (до name mangling'a, т.н. *demangled name*), в библиотеке Boost есть функция, которая это делает:

```
#include <boost/core/demangle.hpp>
#include <iostream>

int main() {
    using namespace boost::core;
    std::cout << demangle(typeid(int).name()) << std::endl;
    std::cout << demangle(typeid(std::string).name()) << std::endl;
}
```

- Все выражения в C++ делятся на два типа: те, кому можно что-то присвоить, и те, кому нельзя

- Все выражения в C++ делятся на два типа: те, кому можно что-то присвоить, и те, кому нельзя
- Те, кому можно присвоить, могут стоять *слева* от оператора присваивания, поэтому называются **lvalue** (left-value)

- Все выражения в C++ делятся на два типа: те, кому можно что-то присвоить, и те, кому нельзя
- Те, кому можно присвоить, могут стоять *слева* от оператора присваивания, поэтому называются **lvalue** (left-value)
- Те, кому нельзя, могут стоять только *справа* от оператора присваивания, поэтому называются **rvalue** (right-value)

- lvalue:

lvalue vs rvalue

- lvalue:
 - Именованные переменные, разыменовывание указателя, обращение по индексу массива, обращение по ссылке

- lvalue:
 - Именованные переменные, разыменовывание указателя, обращение по индексу массива, обращение по ссылке
 - Существуют и вне выражения, в котором использованы

lvalue vs rvalue

- lvalue:
 - Именованные переменные, разыменовывание указателя, обращение по индексу массива, обращение по ссылке
 - Существуют и вне выражения, в котором использованы
 - Имеют адрес в памяти

lvalue vs rvalue

- lvalue:
 - Именованные переменные, разыменовывание указателя, обращение по индексу массива, обращение по ссылке
 - Существуют и вне выражения, в котором использованы
 - Имеют адрес в памяти
- rvalue:

lvalue vs rvalue

- lvalue:
 - Именованные переменные, разыменовывание указателя, обращение по индексу массива, обращение по ссылке
 - Существуют и вне выражения, в котором использованы
 - Имеют адрес в памяти
- rvalue:
 - Временные значения

lvalue vs rvalue

- lvalue:
 - Именованные переменные, разыменовывание указателя, обращение по индексу массива, обращение по ссылке
 - Существуют и вне выражения, в котором использованы
 - Имеют адрес в памяти
- rvalue:
 - Временные значения
 - Уничтожаются после вычисления выражения

lvalue vs rvalue

- lvalue:
 - Именованные переменные, разыменовывание указателя, обращение по индексу массива, обращение по ссылке
 - Существуют и вне выражения, в котором использованы
 - Имеют адрес в памяти
- rvalue:
 - Временные значения
 - Уничтожаются после вычисления выражения
- **NB:** точные правила много сложнее, а ещё есть *xvalue*, *glvalue* и *prvalue*, но их знание не нужно в 99.99% случаев

lvalue vs rvalue

```
int i;

// i - lvalue, 42 - rvalue
i = 42;

int * p = ...;
// p - lvalue, *p - lvalue, i - lvalue
*p = i;

// p[10] - lvalue, p[2] - lvalue, p[2] + 1 - rvalue
p[10] = p[2] + 1;

//      p + 10      rvalue
// *(p + 10)      lvalue
//          i      lvalue
//      i - 5      rvalue
*(p + 10) = i - 5;
```


Move семантика, rvalue-ссылки

- Часто бывают типы, объекты которых не хочется или невозможно копировать: большие динамические массивы, файлы, сетевые сокеты, обёртки других объектов, и т.п.

Move семантика, rvalue-ссылки

- Часто бывают типы, объекты которых не хочется или невозможно копировать: большие динамические массивы, файлы, сетевые сокеты, обёртки других объектов, и т.п.
- Даже если копирование можно реализовать, оно влечёт накладные расходы

Move семантика, rvalue-ссылки

- Часто бывают типы, объекты которых не хочется или невозможно копировать: большие динамические массивы, файлы, сетевые сокеты, обёртки других объектов, и т.п.
- Даже если копирование можно реализовать, оно влечёт накладные расходы

```
vector<int> generate();  
void consume(vector<int> v);  
  
int main() {  
    // 1. Создали временный массив в возвращаемом значении  
    // 2. Скопировали временный массив в v  
    // 3. Удалили временный массив  
    vector<int> v = generate();  
  
    // Скопировали v в аргумент функции  
    consume(v);  
  
    // Удалили v  
}
```

- Хочется уметь *передавать владение* объектом из одного места кода в другое

- Хочется уметь *передавать владение* объектом из одного места кода в другое
- Решение: добавим новый тип ссылок – *rvalue-ссылки* **T&&**

- Хочется уметь *передавать владение* объектом из одного места кода в другое
- Решение: добавим новый тип ссылок – *rvalue-ссылки* **T&&**
- Такая ссылка обычно означает временное значение, которое скоро удалится, и у которого можно забрать данные

- Хочется уметь *передавать владение* объектом из одного места кода в другое
- Решение: добавим новый тип ссылок – *rvalue-ссылки* **T&&**
- Такая ссылка обычно означает временное значение, которое скоро удалится, и у которого можно забрать данные
- При передаче rvalue объектов в функции, они предпочитают перегрузки, принимающие rvalue-ссылку

Move семантика, rvalue-ссылки

```
class Array {
private:
    int * data;
    size_t size;

public:
    Array(): data(nullptr), size(0) {}
    Array(size_t size) : data(new int[size]), size(size) {}

    // Обычное копирование
    Array(const Array& other)
        : data(new int[other.size]), size(other.size)
    { /* std::copy(...) */ }

    // Конструктор от rvalue-ссылки => можем "забрать себе" данные
    Array(Array&& other)
        : data(other.data), size(other.size)
    {
        // Чтобы избежать double free
        other.data = nullptr; other.size = 0;
    }
};
```


Move семантика, rvalue-ссылки

```
Array a1(100);  
  
// Вызов обычного копирования  
Array a2(a1);  
  
// Вызов конструктора от rvalue объекта  
Array a3(A(200));  
Array a4(makeArray());
```

Move семантика, rvalue-ссылки

- Обычно вместе с конструктором пишут оператор присваивания от rvalue:

```
class Array {  
    ...  
  
    Array& operator=(Array&& other) {  
        if (this == &other) return *this;  
        delete data;  
        data = other.data;  
        size = other.size;  
        other.data = nullptr;  
        other.size = nullptr;  
        return *this;  
    }  
};
```

Move семантика, rvalue-ссылки

- Обычно вместе с конструктором пишут оператор присваивания от rvalue:

```
class Array {  
    ...  
  
    Array& operator=(Array&& other) {  
        if (this == &other) return *this;  
        delete data;  
        data = other.data;  
        size = other.size;  
        other.data = nullptr;  
        other.size = nullptr;  
        return *this;  
    }  
};
```

- В общем случае rvalue-ссылки можно использовать где угодно: как типы переменных, как параметры функций, и т.п.

Move семантика, rvalue-ссылки

- Обычно вместе с конструктором пишут оператор присваивания от rvalue:

```
class Array {  
    ...  
  
    Array& operator=(Array&& other) {  
        if (this == &other) return *this;  
        delete data;  
        data = other.data;  
        size = other.size;  
        other.data = nullptr;  
        other.size = nullptr;  
        return *this;  
    }  
};
```

- В общем случае rvalue-ссылки можно использовать где угодно: как типы переменных, как параметры функций, и т.п.
- Обычно они имеют смысл только в специализированных конструкторах и операторах присваивания

- Что, если мы хотим переложить (*move*) один объект в другой без копирования?

Move семантика, rvalue-ссылки

- Что, если мы хотим переложить (*move*) один объект в другой без копирования?
- Стандартная функция `std::move` делает приведение типа из обычной ссылки `T&` в rvalue-ссылку `T&&`

Move семантика, rvalue-ссылки

- Что, если мы хотим переложить (*move*) один объект в другой без копирования?
- Стандартная функция `std::move` делает приведение типа из обычной ссылки `T&` в rvalue-ссылку `T&&`

```
Array a1;  
Array a2(100);  
  
// Вызовет Array::operator=(Array&&)  
a1 = std::move(a2);
```

- У класса можно удалить конструктор и оператор обычного копирования, и оставить только rvalue-версии

Move-only объекты

- У класса можно удалить конструктор и оператор обычного копирования, и оставить только rvalue-версии
- Такой класс нельзя скопировать, можно только переместить (*move*) значение из одного объекта в другой

Move-only объекты

- У класса можно удалить конструктор и оператор обычного копирования, и оставить только rvalue-версии
- Такой класс нельзя скопировать, можно только переместить (*move*) значение из одного объекта в другой
- Такие классы называют *move-only* или *movable-only*

Move-only объекты

- У класса можно удалить конструктор и оператор обычного копирования, и оставить только rvalue-версии
- Такой класс нельзя скопировать, можно только переместить (*move*) значение из одного объекта в другой
- Такие классы называют *move-only* или *movable-only*

```
class Array {  
public:  
    // Запрет копирования  
    Array(const Array&) = delete;  
    Array& operator=(const Array& other) = delete;  
  
    // Move-методы  
    Array(Array&&);  
    Array& operator=(Array&& other);  
};
```

Move семантика, rvalue-ссылки

- Возвращаемое значение функции – автоматически rvalue, поэтому в `return ...`; не нужно делать `std::move` (это ещё и испортит RVO/NRVO)

```
Array makeArray() {  
    Array a(100);  
    // ...  
  
    // Автоматически вызовет rvalue-конструктор, если  
    // не сработало NRVO  
    return a;  
}
```

Move семантика, rvalue-ссылки

- `std::swap` на самом деле реализован через `std::move`, так что его безопасно использовать и для тяжело копируемых объектов (напр. `std::vector`):

```
template <typename T>
void swap(T& a, T& b) {
    T temp = std::move(a);
    a = std::move(b);
    b = std::move(temp);
}
```

Move семантика vs линейные/аффинные типы

- Весь механизм с использованием rvalue-ссылок называется *move-семантикой*

Move семантика vs линейные/аффинные типы

- Весь механизм с использованием rvalue-ссылок называется *move-семантикой*
- Линейный/аффинный тип – термин из теории типов, означающий объекты, которые нужно использовать ровно один раз/не более одного раза

Move семантика vs линейные/аффинные типы

- Весь механизм с использованием rvalue-ссылок называется *move-семантикой*
- Линейный/аффинный тип – термин из теории типов, означающий объекты, которые нужно использовать ровно один раз/не более одного раза
- Относительно новая идея для языков программирования (один из примеров – Rust)

Move семантика vs линейные/аффинные типы

- Весь механизм с использованием rvalue-ссылок называется *move-семантикой*
- Линейный/аффинный тип – термин из теории типов, означающий объекты, которые нужно использовать ровно один раз/не более одного раза
- Относительно новая идея для языков программирования (один из примеров – Rust)
- Должны поддерживаться на уровне компилятора

Move семантика vs линейные/аффинные типы

- Весь механизм с использованием rvalue-ссылок называется *move-семантикой*
- Линейный/аффинный тип – термин из теории типов, означающий объекты, которые нужно использовать ровно один раз/не более одного раза
- Относительно новая идея для языков программирования (один из примеров – Rust)
- Должны поддерживаться на уровне компилятора
- C++ не поддерживает в явном виде линейные или аффинные типы, но позволяет *эмулировать* аффинные типы с помощью move-семантики

Universal references, perfect forwarding

- Есть случай, когда `T&&` не означает rvalue-ссылку: если `T` – шаблонный параметр функции

Universal references, perfect forwarding

- Есть случай, когда **T&&** не означает rvalue-ссылку: если T – шаблонный параметр функции
- Тогда есть особые правила (*reference collapsing rules*), выводящие T как обычную или rvalue-ссылку

Universal references, perfect forwarding

- Есть случай, когда **T&&** не означает rvalue-ссылку: если T – шаблонный параметр функции
- Тогда есть особые правила (*reference collapsing rules*), выводящие T как обычную или rvalue-ссылку
- В этом случае **T&&** называется *универсальной ссылкой* (*universal reference*)

Universal references, perfect forwarding

- Есть случай, когда **T&&** не означает rvalue-ссылку: если T – шаблонный параметр функции
- Тогда есть особые правила (*reference collapsing rules*), выводящие T как обычную или rvalue-ссылку
- В этом случае **T&&** называется *универсальной ссылкой* (*universal reference*)
- Вместе со стандартной функцией **std::forward** позволяет принять неизвестный набор параметров и передать в другую функцию, не потеряв информацию об lvalue/rvalue

Universal references, perfect forwarding

- Есть случай, когда **T&&** не означает rvalue-ссылку: если **T** – шаблонный параметр функции
- Тогда есть особые правила (*reference collapsing rules*), выводящие **T** как обычную или rvalue-ссылку
- В этом случае **T&&** называется *универсальной ссылкой* (*universal reference*)
- Вместе со стандартной функцией **std::forward** позволяет принять неизвестный набор параметров и передать в другую функцию, не потеряв информацию об lvalue/rvalue
- Полезно для методов-фабрик, callback'ов, обёрток, и т.п.

Universal references, perfect forwarding

- Есть случай, когда **T&&** не означает rvalue-ссылку: если **T** – шаблонный параметр функции
- Тогда есть особые правила (*reference collapsing rules*), выводящие **T** как обычную или rvalue-ссылку
- В этом случае **T&&** называется *универсальной ссылкой* (*universal reference*)
- Вместе со стандартной функцией **std::forward** позволяет принять неизвестный набор параметров и передать в другую функцию, не потеряв информацию об lvalue/rvalue
- Полезно для методов-фабрик, callback'ов, обёрток, и т.п.
- Сам механизм называется *perfect forwarding*

Universal references, perfect forwarding

```
template <typename T>  
void foo(T&& x) {  
    bar(std::forward<T>(x));  
}
```

Universal references, perfect forwarding

```
template <typename T>
void foo(T&& x) {
    bar(std::forward<T>(x));
}
```

- Часто используется вместе с variadic templates:

```
template <typename T, typename ... Args>
std::shared_ptr<T> std::make_shared(Args&& ... args) {
    return std::shared_ptr<T>(
        new T(std::forward<Args>(args)...);
    );
}
```

- Часто не хочется писать слишком длинный тип переменной, или его тяжело получить

- Часто не хочется писать слишком длинный тип переменной, или его тяжело получить

```
for (std::map<std::string, std::vector<int>>::const_iterator  
    it = m.begin(); it != m.end(); ++it) { ... }
```

- Часто не хочется писать слишком длинный тип переменной, или его тяжело получить

```
for (std::map<std::string, std::vector<int>>::const_iterator  
    it = m.begin(); it != m.end(); ++it) { ... }
```

```
template <typename Iterator>  
void qsort(Iterator being, Iterator end) {  
    typename std::iterator_traits<Iterator>::value_type  
        pivot = ...;  
    ...  
}
```

- Часто не хочется писать слишком длинный тип переменной, или его тяжело получить

```
for (std::map<std::string, std::vector<int>>::const_iterator  
    it = m.begin(); it != m.end(); ++it) { ... }
```

```
template <typename Iterator>  
void qsort(Iterator being, Iterator end) {  
    typename std::iterator_traits<Iterator>::value_type  
        pivot = ...;  
    ...  
}
```

- Решение: `auto`

- Ключевое слово `auto` можно использовать вместо типа переменной, тогда тип выведется автоматически из выражения, которым инициализируется переменная

Вывод типов

- Ключевое слово **auto** можно использовать вместо типа переменной, тогда тип выведется автоматически из выражения, которым инициализируется переменная

```
// int
auto i = 10;

// const char*
auto str = "Hello, world!";

std::map<std::string, std::vector<int>> m;

// std::map<std::string, std::vector<int>>::iterator
auto it = m.begin();

// std::pair<const std::string, std::vector<int>>
// NB: не ссылка, а копия!
auto pivot = *it;
```


- К `auto` можно добавить спецификаторы, уточняющие тип:

- К **auto** можно добавить спецификаторы, уточняющие тип:

```
int * p = ...;

// int
auto x = *p;

// const int
const auto x = *p;

// int&
auto& x = *p;
```

- Оператор `decltype(x)` подставляет вместо себя тип выражения `(x)`

- Оператор `decltype(x)` подставляет вместо себя тип выражения `(x)`
- Можно использовать везде, где ожидается тип

Вывод типов

- Оператор `decltype(x)` подставляет вместо себя тип выражения (`x`)
- Можно использовать везде, где ожидается тип

```
// int
decltype(10) i = 20;

// float
decltype(1 + 2.f) f = 3.14f;

int* p = ...;

// int*
decltype(p + 10) q = ...;

// int&
decltype(p[10]) r = ...;
```

ВЫВОД ТИПОВ

- Оператор `decltype(x)` подставляет вместо себя тип выражения (`x`)
- Можно использовать везде, где ожидается тип

```
// int
decltype(10) i = 20;

// float
decltype(1 + 2.f) f = 3.14f;

int* p = ...;

// int*
decltype(p + 10) q = ...;

// int&
decltype(p[10]) r = ...;
```

- NB: есть фокусы связанные со ссылками, см. `decltype(auto)` и `decltype((x))`

- `decltype` часто полезен в сложном шаблонном коде

- `decltype` часто полезен в сложном шаблонном коде
- `auto` нужно использовать аккуратно

- `decltype` часто полезен в сложном шаблонном коде
- `auto` нужно использовать аккуратно
- Где-то `auto` упрощает чтение кода (напр. с итераторами – мы и так знаем, что там за тип)

- `decltype` часто полезен в сложном шаблонном коде
- `auto` нужно использовать аккуратно
- Где-то `auto` упрощает чтение кода (напр. с итераторами – мы и так знаем, что там за тип)
- Где-то `auto` усложняет чтение кода (непонятно, какая переменная в итоге какого типа)

Range-based for loop

- Даже с итераторами, простая концепция ‘пройтись по всем элементам коллекции’ выглядит довольно громоздко:

```
for (auto it = v.begin(); it != v.end(); ++it) {  
    int x = *it;  
    // ...  
}
```

Range-based for loop

- Даже с итераторами, простая концепция ‘пройтись по всем элементам коллекции’ выглядит довольно громоздко:

```
for (auto it = v.begin(); it != v.end(); ++it) {  
    int x = *it;  
    // ...  
}
```

- Решение: *range-based for loop*:

```
for (int x : v) {  
    // ...  
}
```

Range-based for loop

- Даже с итераторами, простая концепция ‘пройтись по всем элементам коллекции’ выглядит довольно громоздко:

```
for (auto it = v.begin(); it != v.end(); ++it) {  
    int x = *it;  
    // ...  
}
```

- Решение: *range-based for loop*:

```
for (int x : v) {  
    // ...  
}
```

- Превращается компилятором в точно такой же код с итераторами

Range-based for loop

- Как при объявлении переменной, вместо `int` можно написать `auto` или принимать по ссылке:

```
std::vector<int> v;  
  
// Локальный x - копия элемента коллекции  
for (int x : v) { ... }  
  
// Тип выведется как int  
for (auto x : v) { ... }  
  
// Локальный x - ссылка на элемент коллекции  
for (int& x : v) { ... }  
  
// Тип выведется как int&  
for (auto& x : v) { ... }
```

Range-based for loop

- Если бы такой цикл использовал *методы* `begin()` и `end()`, то он бы не работал с обычными массивами:

```
int array[5];  
for (int v : array) { ... }
```

Range-based for loop

- Если бы такой цикл использовал *методы* `begin()` и `end()`, то он бы не работал с обычными массивами:

```
int array[5];  
for (int v : array) { ... }
```

- На самом деле, вместо этого range-based for использует стандартные функции `std::begin()` и `std::end()`

Range-based for loop

- Если бы такой цикл использовал *методы* `begin()` и `end()`, то он бы не работал с обычными массивами:

```
int array[5];  
for (int v : array) { ... }
```

- На самом деле, вместо этого range-based for использует стандартные функции `std::begin()` и `std::end()`
- Для обычных массивов они возвращают нужные указатели, а иначе вызывают метод `v.begin()` или `v.end()`